Lab: Exploring C Compilation Across Instruction Set Architectures with Compiler Explorer

Duration: approximately 2 hours **Tool:** Compiler Explorer (Godbolt) **Language:** C

Objective

In this exercise, you will use Compiler Explorer to investigate how a simple C program is translated into machine instructions for different **instruction set architectures (ISAs)**.

You will compare the same C programs compiled for different architectures, such as:

- x86-64
- ARM64 / AArch64
- RISC-V

The goal is **not** to learn assembly programming.

Instead, you should learn to answer:

> **How does the ISA influence the way a compiler represents a simple C computation?**

You will progressively investigate:

1. integer constants and moves;
2. addition and subtraction;
3. multiplication and division;
4. Boolean and comparison operations;
5. `if` statements.

Do not consider compiler optimization flags in this exercise. Keep the compiler configuration fixed when comparing architectures.

1. Integer constants and moves

Start with the following C program:

```
int get_value(void)
{
    return 42;
}
```

Compile it for x86-64, ARM64, and RISC-V.

Questions

Look at the generated assembly.

1. How is the value `42` introduced?
2. Which register contains the return value?
3. Is there an instruction that explicitly moves `42` into a register?
4. How many instructions are required?
5. Are the three architectures using the same instruction format?

Create a table:

	x86-64	ARM64	RISC-V
Instruction(s)			
Return-value register			
Number of instructions			

Try another constant

Change the program to:

```
int get_value(void)
{
    return 1000;
}
```

Then try:

```
int get_value(void)
{
    return 100000;
}
```

Questions

1. Does the generated code change?
2. Can every ISA encode an arbitrary integer constant directly in one instruction?
3. If not, what does the compiler do?

This introduces an important ISA concept:

> **Instructions have a limited encoding. An ISA must specify how constants can be represented inside instructions.**

---

2. Addition and subtraction

Consider:

```
int add(int a, int b)
{
    return a + b;
}
```

Compile it for the three architectures.

Questions

Identify:

- the registers containing `a` and `b`;
- the instruction performing the addition;
- the register containing the result.

Then consider:

```
int subtract(int a, int b)
{
    return a - b;
}
```

Compare the two programs.

Questions

1. What instruction performs addition?
2. What instruction performs subtraction?
3. How many operands does each instruction have?
4. Is the destination register also one of the input registers?
5. Do all three ISAs use the same operand structure?

Three operands or two?

Pay particular attention to examples where an instruction looks conceptually like:

```
destination = source1 + source2
```

versus:

```
destination = destination + source
```

Determine which model is used by each architecture.

This is an important ISA-level difference.

---

3. Combining arithmetic operations

Now try:

```
int calculate(int a, int b)
{
    return a + b - 1;
}
```

Then:

```
int calculate(int a, int b)
{
    return a - b + 1;
}
```

Questions

1. How many arithmetic instructions are generated?
2. Which intermediate values are kept in registers?
3. Does each architecture use the same number of registers?
4. Does the instruction syntax make the data flow easy or difficult to see?

Try to reconstruct the computation from the assembly.

For example:

```
a ──────┐
        +──> result
b ──────┘
```

---

4. Multiplication

Consider:

```
int multiply(int a, int b)
{
    return a * b;
}
```

Compare x86-64, ARM64, and RISC-V.

Questions

1. Does each ISA have a multiplication instruction?
2. How many operands does it take?
3. Where is the result placed?
4. Does multiplication have the same instruction structure as addition?
5. Are there any important differences between the architectures?

Now try:

```
int multiply_by_two(int x)
{
    return x * 2;
}
```

and:

```
int multiply_by_ten(int x)
{
    return x * 10;
}
```

Questions

Look at the generated instructions.

1. How is the constant represented?
2. Is multiplication by a constant treated differently?
3. Does the ISA provide any special instruction that makes this convenient?

Do not try to explain the compiler's choice in terms of optimization.

Focus on:

> **What operations does the ISA make available?**

5. Division

Now consider:

```
int divide(int a, int b)
{
    return a / b;
}
```

Compare the three architectures.

Questions

1. Does every ISA provide an integer division instruction?
2. What is the instruction called?
3. How many operands does it use?
4. Where is the quotient placed?
5. Is division structurally similar to multiplication?

Now try:

```
int divide_by_two(int x)
{
    return x / 2;
}
```

Compare the generated code.

Signed versus unsigned division

Try these two programs:

```
int divide(int a, int b)
{
    return a / b;
}
```

and:

```
unsigned int divide(unsigned int a, unsigned int b)
{
    return a / b;
}
```

Questions

1. Does the generated assembly differ?
2. Does the ISA distinguish signed and unsigned division?
3. If so, how?

This is a good example of the interaction between:

```
C types
   ↓
C semantics
   ↓
ISA instructions
```

---

6. Boolean values

Now move from arithmetic to Boolean values.

Consider:

```
int always_true(void)
{
    return 1;
}
```

and:

```
int always_false(void)
{
    return 0;
}
```

Questions

1. How does C represent `true` and `false` as integers?
2. How does each ISA return these values?
3. Are there special Boolean registers?
4. Or are Boolean values ordinary integer values?

---

7. Comparisons

Consider:

```
int equal(int a, int b)
{
    return a == b;
}
```

Then:

```
int less(int a, int b)
{
    return a < b;
}
```

And:

```
int greater(int a, int b)
{
    return a > b;
}
```

Compare all three architectures.

Questions

Pay particular attention to the **comparison mechanism**.

1. Does the comparison directly produce a value?

2. Does it modify special processor state?
3. What instruction performs the comparison?
4. How does the result become the C value `0` or `1`?
5. Is the mechanism the same on x86-64, ARM64, and RISC-V?

Important comparison

This is one of the most important parts of the exercise.

Compare how the following C expression is implemented:

```
return a < b;
```

Try to describe the three implementations at a conceptual level.

For example:

```
C:

        a < b
          |
          v
        0 or 1
```

Then describe what happens on each ISA.

Pay special attention to the difference between:

> **comparison producing a Boolean value**

and:

> **comparison establishing a condition that another instruction can use.**

---

8. `if` statements

Now use the comparison in a control-flow context.

```
int abs_value(int x)
{
    if (x < 0)
        return -x;

    return x;
}
```

Compare the three architectures.

Try to identify:

1. the comparison;
2. the conditional branch;
3. the negation;
4. the two possible return paths.

Draw the control flow:

```
            x < 0?
            /   \
          yes    no
           |      |
          -x      x
            \    /
             return
```

9. A simple `if/else`

Now consider:

```
int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}
```

Compare x86-64, ARM64, and RISC-V.

Questions

1. How is the condition evaluated?
2. How is the branch performed?
3. What instruction transfers control to the `else` case?
4. What instruction transfers control to the end?
5. How does the architecture represent the condition used by the branch?

Do not focus on whether the generated code is "good" or "bad".

Instead ask:

> **What does this ISA provide for implementing conditional control flow?**

---

10. Combining comparison and arithmetic

Consider:

```
int clamp_positive(int x)
{
    if (x < 0)
        return 0;

    return x;
}
```

Then:

```
int increment_if_positive(int x)
{
    if (x > 0)
        return x + 1;

    return x;
}
```

Compare the three ISAs.

Questions

For each architecture, identify the sequence:

```
       input
         |
         v
     comparison
         |
         v
      condition
         |
         v
       branch
        /   \
       /     \
   path A  path B
       \     /
        result
```

Which parts of this structure are common to all three architectures?

Which parts are ISA-specific?

---

11. ISA detective work

You have now seen several different operations:

- constants;

- addition;
- subtraction;
- multiplication;
- division;
- comparison;
- conditional branching.

Choose **one operation** and investigate it in the official ISA documentation for all three architectures.

For example:

> How does RISC-V perform an integer comparison?

or:

> How does ARM64 perform a conditional branch?

or:

> How does x86-64 perform an integer comparison?

For each architecture, record:

	x86-64	ARM64	RISC-V
Relevant instruction(s)			
Number of operands			
Result			
Special state involved?			
Can it branch directly?			

12. Final challenge — Same C, three ISAs

Consider:

```
int f(int a, int b)
{
    if (a < b)
        return a + 1;
    else
        return a - 1;
}
```

Before looking at the assembly, predict what the compiler will need:

```

1. compare a and b
2. decide which branch to take
3. perform either:
       a + 1
   or:
       a - 1

4. return the result
```

Now inspect the output for:

- x86-64;
- ARM64;
- RISC-V.

For each architecture, annotate the assembly with comments showing which C operation each instruction contributes to.

---

13. What did the ISA change?

Look back at all the examples.

Complete the following table:

C operation	x86-64	ARM64	RISC-V
Load a constant			
Add two integers			
Subtract two integers			
Multiply two integers			
Divide two integers			
Compare two integers			
Conditional branch			

Then answer:

Question 1

Which operations look very similar across all three architectures?

Question 2

Which operations reveal the largest differences?

Question 3

Which differences are caused by the **instruction format**?

Question 4

Which differences are caused by the **way the ISA represents conditions**?

Question 5

Why can three completely different sequences of machine instructions all correctly implement the same C program?

Deliverable

Submit a short report containing:

1. Three architectures

For each architecture, give a brief description of its general approach to:

- arithmetic;
- registers;
- comparisons;
- conditional branches.

2. Two comparisons

Choose **two C examples** from the exercise.

For each one, show:

- the C source;
- the generated assembly for the three ISAs;
- annotations identifying the important instructions;
- a short explanation of the differences.

3. One interesting ISA difference

Describe one difference between x86-64, ARM64, and RISC-V that you found particularly interesting.

Explain how that difference affects the generated assembly.

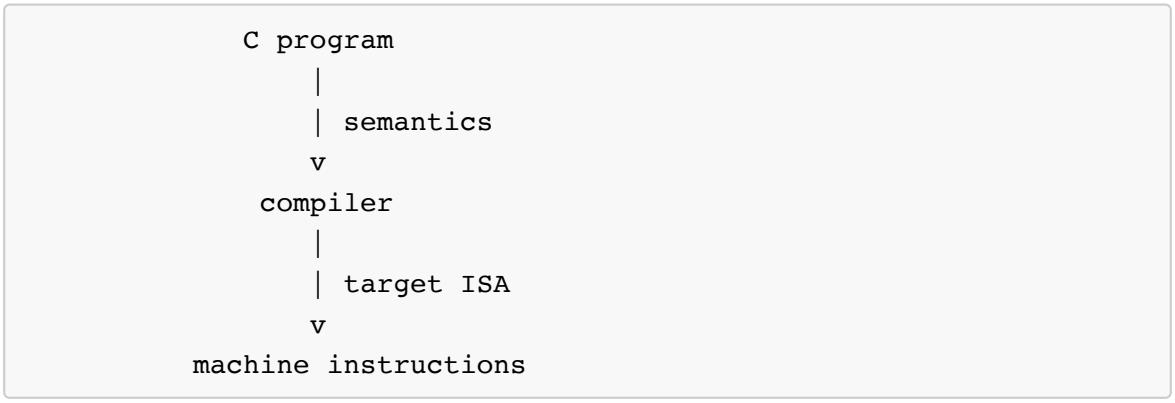
4. Final reflection

Answer:

> **If the C program is the same, why does the generated assembly depend so strongly on the target ISA?**

Key idea

Throughout the exercise, keep this model in mind:



The C program says **what computation must be performed**.

The ISA defines **which instructions and mechanisms are available** to perform that computation.

The compiler has to bridge the two.

Your objective is to discover that bridge by looking at very small examples.